

МИНОБРНАУКИ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА
ВЕЛИКОГО»**

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление 02.03.01 Математика и компьютерные науки

Отчёт по дисциплине «Математическая логика и теория автоматов»

Лабораторная работа №1 «Разработка
лексического анализатора»

Вариант №18

Студент: _____

Черданцев Артём Сергеевич

Преподаватель: _____

Востров Алексей Владимирович

«____» _____ 20__ г.

Санкт-Петербург, 2025

Содержание

Введение	3
1 Математическое описание	4
1.1 Конечный автомат-распознаватель	4
1.2 Лексический анализ	4
1.3 Лексемы заданного языка	4
1.4 Конечный автомат	5
2 Особенности реализации	7
2.1 Класс lab01	7
2.2 Класс Token	7
2.3 Функция isSeparator	7
2.4 Функция addToken	7
3 Результаты работы программы	9
Заключение	10
Список источников	11
Приложение 1	12

Введение

В данной лабораторной работе требуется реализовать программу, которая выполняет лексический анализ входного текста в соответствии с заданными правилами и порождает таблицу лексем с указанием их типов и значений. Текст на входном языке задается в виде символьного (текстового) файла. Программа должна выдавать сообщения о наличии во входном тексте ошибок, которые могут быть обнаружены на этапе лексического анализа. Длина идентификатора и строковых констант ограничена 16 символами. Программа должна допускать наличие комментариев неограниченной длины во входном файле (форма комментариев выбирается самостоятельно). В моем варианте введены следующие правила языка:

- . Входной язык содержит логические выражения, разделенные символом ; (точка с запятой). Логические выражения состоят из идентификаторов, констант **T** и **F**, знака присваивания ($:=$), операций **or**, **xor**, **and**, **not**, **nand**, **nor**, знаки сравнения $<$, $>$, $=$ (а также их комбинаций) и круглых скобок
- Комментарии начинаются со спец. символа $//$ и имеют неограниченную длину в рамках 1 строки.

1 Математическое описание

1.1 Конечный автомат-распознаватель

Конечный автомат-распознаватель — пятерка объектов $A = \langle S, X, s_0, \sigma, F \rangle$, где:

- $S \neq \emptyset$ - конечное множество состояний;
- $X \neq \emptyset$ - конечное множество входных сигналов (входной алфавит);
- $s_0 \in S$ - множество начальных состояний;
- $\sigma : S \times X \rightarrow S$ - функция переходов;
- $F \subset S$ - множество финальных состояний.

Обобщенная функция переходов $\sigma^* : S \times X^* \rightarrow S$ определяется так:

$$\begin{aligned}\sigma^*(s, \epsilon) &= s; \\ \sigma^*(s, a\alpha) &= \sigma^*(\sigma(s, a)\alpha) \quad \forall s \in S, a \in X, \alpha \in X^*\end{aligned}$$

Конечный автомат-распознаватель A *допускает входную цепочку* $\alpha \in X^*$, если α переводит его из начального состояния в одно из конечных. Множество всех цепочек, допускаемых автоматом A , образует язык L_A , допускаемый этим автоматом.

$$L_A = \{\alpha \in X^* : \sigma^*(s_0, \alpha) \in F\}$$

Конечный автомат-транслятор — конечный автомат-распознаватель, дополненный семантическими действиями на переходах — $A = \langle S, X, Y, s_0, \sigma, \lambda, F \rangle$, где:

- Y - конечное множество семантических действий;
- $\lambda^S : S \times X \rightarrow Y^*$ — функция выходов.

1.2 Лексический анализ

Конечные автоматы используются также в области трансляции языков, то есть преобразовании цепочки в некоторый выход. Если цепочка не является цепочкой языка, то выдается информация об ошибке, иначе требуемый результат.

Лексический анализатор читает поток символов, составляющих исходную программу, и группирует эти символ в значащие последовательности (лексемы).

Лексема — это структурная единица языка, которая состоит из элементарных символов языка и не содержит в своём составе других структурных единиц языка. Лексемами языков программирования являются идентификаторы, константы, ключевые слова языка, знаки операций и т.п.

На вход транслятора поступает исходная программа. На выходе лексического анализатора программа представлена в виде последовательности лексем (в данной реализации — таблицы).

1.3 Лексемы заданного языка

Язык, допускаемый анализатором в данной работе, содержит следующие типы лексем:

- идентификатор — последовательность латинских букв (в верхнем и нижнем регистре) и цифр $(0, \dots, 9)$. Первый символ должен быть буквой. Максимальная длина — 16 символов.
- знак присваивания — $:=$;

- знаки операций: +, -, * и /;
- скобки: (и);
- константы T и F;
- операции or, xor, and, not, nand, nor;
- знаки сравнения <, >, =.

Анализатором игнорируются комментарии. Комментарий начинается со спец. символа // и имеет неограниченную длину в рамках 1 строки.

1.4 Конечный автомат

Лексический анализатор был реализован на основе конечного автомата (Рис. 1). Состояние E — незаключительное состояние, в которое этот автомат переходит из остальных состояний при подаче недопустимых входов, причем под воздействием всех входных сигналов автомат не выходит из него. На рисунке оно не отражено.

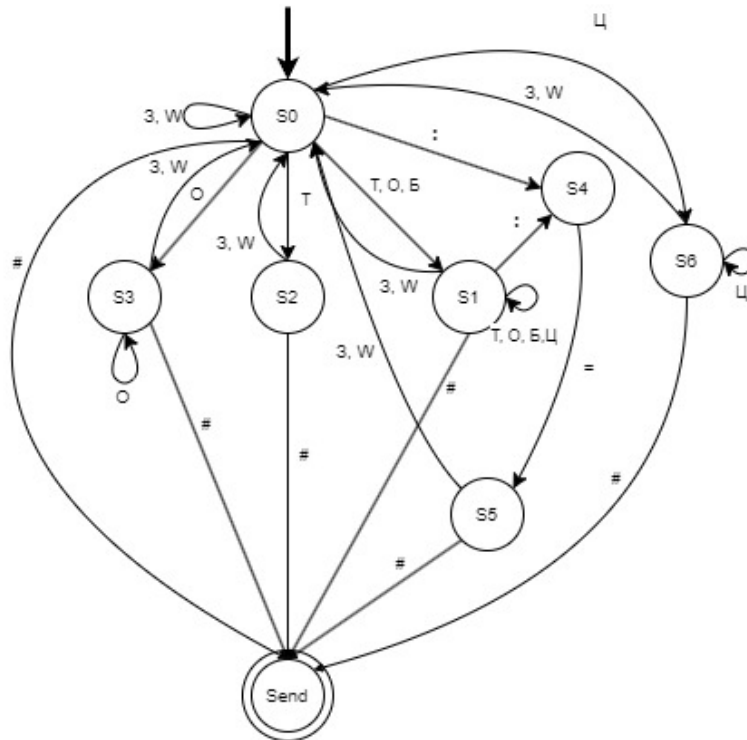


Рис. 1: Детерминированный конечный автомат

Формальное определение автомата:

$$A = \langle S, X, Y, s_0, \sigma', F \rangle,$$

где $S = \{0, 1, 2, 3, 4, 5, 6, end, E\}$,

$$F = \{S_{end}\},$$

$X = \{X_i : i \in \{1, \dots, 9\}\} = \{\#, \text{Ц}, \text{О}, \text{Т}, \text{Б}, \text{З}, \text{W}, :, =\}$, где:

- $\# = \{-1\}$ - символ окончания ввода;

- $\mathbb{C} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ - цифры;
- $\mathbb{O} = \{n, o, t, r, a, n, d, x\}$ - буквы, используемые в логических операциях;
- $\mathbb{T} = \{T, F\}$ - буквы, используемые для обозначения Т и F констант;
- $\mathbb{B} = \{b, c, \dots, z, A, B, \dots, Z\}$ - остальные буквы;
- $\mathbb{Z} = \{(\,,\,), +, *, /, ;, <, >\}$ — знаки операций из одного символа;
- $\mathbb{W}$ — множество пробельных символов (пробел, табуляция, перенос строки и прочие);
- $:, =$ — обозначают самих себя.

$\sigma' : S \times X' \rightarrow S$ - функция переходов, которая порождается функцией $\sigma : S \times X \rightarrow S$ по правилу

$$(\forall s \in S, x \in X')(\exists! X_i \in X)\sigma'(s, x) = \sigma(X_i), \text{ где } x \in X_i \subset X'$$

σ задана таблицей (Рис. 2)

σ	#	$\mathbb{C}$	$\mathbb{O}$	$\mathbb{T}$	$\mathbb{B}$	$\mathbb{Z}$	$\mathbb{W}$	:	=
0	end	6	1,3	1,2	1	0	0	4	E
1	end	1	1	1	1	0	0	4	E
2	end	E	E	E	E	0	0	E	E
3	end	E	3	E	E	0	0	E	E
4	E	E	E	E	E	E	E	E	5
5	end	E	E	E	E	0	0	E	E
6	end	6	E	E	E	0	0	E	E
end	E	E	E	E	E	E	E	E	E
E	E	E	E	E	E	E	E	E	E

Рис. 2: Таблица переходов

2 Особенности реализации

Для реализации конечного автомата была написана программа на Java.

Код программы и всех функций см. в Приложение 1.

2.1 Класс lab01

Класс lab01 - основной класс программы, представляет собой лексический анализатор. Классу передаётся поток символов (класс Reader), представляющий собой текстовый файл, содержащий код, который будет анализироваться.

Во время итерирования может возникнуть исключение UnreadableCharacterException.

2.2 Класс Token

Класс Token – класс, представляющий лексему.

В нём объявлены поля:

- value: строка, содержащая значение лексемы.
- type: строка, представляющая тип лексемы.
- tokenValue: строка, дополнительное значение лексемы (например, значение переменной).

2.3 Функция isSeparator

Функция проверяет, является ли переданный символ разделителем (символом конца оператора) или же скобками.

Вход: Символ.

Выход: "True если символ является разделителем, иначе - "False".

```
1 private static boolean isSeparator(char symbol) {  
2     return symbol == ';' || symbol == '(' || symbol == ')' || symbol == '=';  
3 }
```

2.4 Функция addToken

Функция добавляет лексему в список tokenList в зависимости от её значения. Определяет тип лексемы и, при необходимости, её дополнительные значения (например, значение переменной).

Если же тип лексемы определить не удалось, то она добавляется в tokenList со значениями: **Тип** - Неизвестный, **Значение** - ∅.

Вход: Строковое значение лексемы.

Выход: Лексема с присвоенным ей типом.

```
1 private static void addToken(String value, List<Token> tokenList, Map<String,  
    ↪ Integer> variableMap, int[] keywordCount, int[] varCount) {  
2     if (!value.isEmpty()) {
```

```

3      switch (value) {
4          case "(":
5              tokenList.add(new Token(value, "Знак присваивания", "("));
6              break;
7          case "T":
8              tokenList.add(new Token(value, "Булева константа", "True"));
9              break;
10         case "F":
11             tokenList.add(new Token(value, "Булева константа", "False"));
12             break;
13         case "begin":
14         case "end":
15             tokenList.add(new Token(value, "Ключевое слово", "X" +
16                 ↪ keywordCount[0]++));
17             break;
18         case "<":
19         case ">":
20         case "<=":
21         case ">=":
22         case "==" :
23             tokenList.add(new Token(value, "Знак сравнения", "="));
24             break;
25         case "or":
26         case "xor":
27         case "and":
28         case "not":
29         case "nand":
30         case "nor":
31             tokenList.add(new Token(value, "Логическая операция", ""));
32             break;
33         default:
34             if (isNumeric(value)) {
35                 tokenList.add(new Token(value, "Вещественная константа",
36                     ↪ value));
37             } else if (value.matches("[a-zA-Z]+") && keywordCount[0] >= 1) {
38                 if (!variableMap.containsKey(value)) {
39                     variableMap.put(value, varCount[0]);
40                     tokenList.add(new Token(value, "Переменная", value + ":" +
41                         ↪ varCount[0]++));
42                 }
43             } else {
44                 tokenList.add(new Token(value));
45             }
46             break;
47     }
48 }

```

3 Результаты работы программы

На Рис. 3 представлены результаты работы программы с входным файлом, в котором рассматриваются все основные варианты ошибок и правильных лексем.

Для анализа был взят следующий пример кода:

```
1 //lab01
2 begin
3   x :=T;
4   y := F andand F;
5   z ::= x and y;
6   begi
7     resuasdasdasdasdasdasdlt := (x > y) or (z and not(x));
8   end;
9   x := 13a;
10 end
```

Ошибка! Неверная лексема: =

Ошибка! Неизвестная лексема: :

Ошибка! Лексема resuasdasdasdasdasdasdlt имеет длину более 16 символов!

Ошибка! Неизвестная лексема: 13a

Лексема	Тип	Значение
begin	Ключевое слово	X1
x	Переменная	x:1
:=	Знак присваивания	
T	Булевая константа	True
;	Символ	
y	Переменная	y:2
:=	Знак присваивания	
F	Булевая константа	False
andand	Переменная	andand:3
F	Булевая константа	False
;	Символ	
z	Переменная	z:4
and	Логическая операция	
;	Символ	
begi	Переменная	begi:5
:=	Знак присваивания	
(	Символ	S0
>	Знак сравнения	
)	Символ	S1
or	Логическая операция	
(	Символ	S0
and	Логическая операция	
not	Логическая операция	
(	Символ	S0
)	Символ	S1
)	Символ	S1
;	Символ	
end	Ключевое слово	X2
;	Символ	
:=	Знак присваивания	
;	Символ	
end	Ключевое слово	X3

Рис. 3: Результат работы программы

Заключение

Результатом выполнения работы стала программа, лексический анализатор, созданный с помощью конечного автомата для распознавания лексем, формирующий соответствующие таблицы с верными лексемами и ошибочными с указанием типов и значений лексем.

Преимущества:

- Два типа обработки некорректных лексем;
- Программа продолжает работу при достижении ошибки.

Недостатки:

- Результаты работы программы выводятся на консоль в человекочитаемом формате, что не позволяет использовать ее совместно с другими программами.
- Сложная адаптация кода для других входных языков

Масштабируемость:

- Считывание строк из консоли;
- Запись таблицы лексем в файл.

Список источников

1. Карпов, Ю.Г. Теория автоматов. – СПб.: Питер, 2003. – С. 208.
2. Карпов, Ю.Г. Теория и технология программирования. Основы построения трансляторов. – СПб.: БХВ-Петербург, 2005. – С. 272.
3. Курс лекций по математической логике и теории автоматов. Востров А.В. URL: https://tema.spbstu.ru/t_automata/ (Дата обращения: 15.04.2024)

Приложение 1

```
1  import java.io.FileReader;
2  import java.io.IOException;
3  import java.io.PushbackReader;
4  import java.util.ArrayList;
5  import java.util.HashMap;
6  import java.util.List;
7  import java.util.Map;
8
9  public class lab01 {
10
11      public static void main(String[] args) {
12          String fileName = "input.txt";
13          List<Token> tokenList = new ArrayList<>();
14          Map<String, Integer> variableMap = new HashMap<>();
15          int[] keywordCount = {1};
16          int[] varCount = {1};
17
18          try (PushbackReader reader = new PushbackReader(new FileReader(fileName)))
19              ↪ {
20              StringBuilder currentToken = new StringBuilder();
21              boolean inComment = false;
22
23              while (true) {
24                  int currentChar = reader.read();
25
26                  if (currentChar == -1) {
27                      break;
28                  }
29
30                  char currentSymbol = (char) currentChar;
31
32                  if (currentSymbol == '/' && reader.ready()) {
33                      char nextChar = (char) reader.read();
34                      if (nextChar == '/') {
35                          inComment = true;
36                          continue;
37                      } else {
38                          reader.unread(nextChar);
39                      }
40                  }
41
42                  if (inComment) {
43                      if (currentSymbol == '\n') {
44                          inComment = false;
45                      }
46                      continue;
47                  }
48              }
49          }
```

```

48     if (Character.isWhitespace(currentSymbol) ||
49         ↪ isSeparator(currentSymbol)) {
50         if (currentToken.length() > 0) {
51             if (currentToken.length() <= 16) {
52                 addToken(currentToken.toString(), tokenList,
53                     ↪ variableMap, keywordCount, varCount);
54                 currentToken = new StringBuilder();
55             } else {
56                 System.out.printf("\nОшибка! Лексема " +
57                     ↪ currentToken.toString() + " имеет длину более 16
58                     ↪ СИМВОЛОВ!");
59                 currentToken = new StringBuilder();
60                 continue;
61             }
62         }
63
64         if (isSeparator(currentSymbol)) {
65             addSeparatorToken(currentSymbol, tokenList);
66         }
67
68     } else {
69         currentToken.append(currentSymbol);
70     }
71
72     if (currentSymbol == ':' && reader.ready()) { //:
73         char nextChar = (char) reader.read();
74         if (nextChar == '=') { //:=
75             currentToken.append(nextChar);
76             addToken(currentToken.toString(), tokenList, variableMap,
77                 ↪ keywordCount, varCount);
78             currentToken = new StringBuilder();
79         } else if (nextChar == ':') { //::
80             char nextNextChar = (char) reader.read();
81             if (nextNextChar == '=') { //::=
82                 System.out.println("\nОшибка! Неверная лексема: " +
83                     ↪ currentToken.toString() + nextChar +
84                     ↪ nextNextChar);
85                 System.out.println("\nОшибка! Неверная лексема: ::=");
86             } else {
87                 System.out.println("\nОшибка! Неверная лексема: " +
88                     ↪ currentToken.toString() + nextChar);
89                 reader.unread(nextNextChar);
90             }
91         } else if (nextChar == '=') { //:=
92             char nextNextChar = (char) reader.read();
93             if (nextNextChar == '=') { //:=
94                 System.out.println("\nОшибка! Неверная лексема: " +
95                     ↪ currentToken.toString() + nextChar +
96                     ↪ nextNextChar);
97             } else {
98

```

```

88         addToken(":", tokenList, variableMap, keywordCount,
89             ↪ varCount);
90         reader.unread(nextNextChar);
91     } else {
92         reader.unread(nextChar);
93     }
94 }
95 }
96
97 if (currentToken.length() > 0) {
98     if (currentToken.length() <= 16) {
99         addToken(currentToken.toString(), tokenList, variableMap,
100             ↪ keywordCount, varCount);
101     }
102 }
103
104 } catch (IOException e) {
105     System.err.println("Ошибка при чтении файла: " + e.getMessage());
106     return;
107 }
108
109 System.out.printf("\n\n");
110 System.out.printf("%-40s%-40s%-40s\n\n", "Лексема", "Тип", "Значение");
111 for (Token token : tokenList) {
112     System.out.printf("%-40s%-40s%-40s\n", token.value, token.getType(),
113         ↪ token.getValue());
114 }
115 System.out.printf("\n\n");
116 }
117
118 private static boolean isSeparator(char symbol) {
119     return symbol == ';' || symbol == '(' || symbol == ')' || symbol == '=';
120 }
121
122 private static void addSeparatorToken(char symbol, List<Token> tokenList) {
123     switch (symbol) {
124         case ';':
125             tokenList.add(new Token(";", "Символ", ""));
126             break;
127         case '(':
128             tokenList.add(new Token("(", "Символ", "S0"));
129             break;
130         case ')':
131             tokenList.add(new Token(")", "Символ", "S1"));
132             break;
133         default:
134             break;
135     }
136 }

```

```

135
136 private static void addToken(String value, List<Token> tokenList, Map<String,
    ↪ Integer> variableMap, int[] keywordCount, int[] varCount) {
137     if (!value.isEmpty()) {
138         switch (value) {
139             case "(":
140                 tokenList.add(new Token(value, "Знак присваивания", ""));
141                 break;
142             case "T":
143                 tokenList.add(new Token(value, "Булевая константа", "True"));
144                 break;
145             case "F":
146                 tokenList.add(new Token(value, "Булевая константа", "False"));
147                 break;
148             case "begin":
149             case "end":
150                 tokenList.add(new Token(value, "Ключевое слово", "X" +
    ↪ keywordCount[0]++));
151                 break;
152             case "<":
153             case ">":
154             case "<=":
155             case ">=":
156             case "==":
157                 tokenList.add(new Token(value, "Знак сравнения", ""));
158                 break;
159             case "or":
160             case "xor":
161             case "and":
162             case "not":
163             case "nand":
164             case "nor":
165                 tokenList.add(new Token(value, "Логическая операция", ""));
166                 break;
167             default:
168                 if (isNumeric(value)) {
169                     tokenList.add(new Token(value, "Вещественная константа",
    ↪ value));
170                 } else if (value.matches("[a-zA-Z]+") && keywordCount[0] >= 1)
    ↪ {
171                     if (!variableMap.containsKey(value)) {
172                         variableMap.put(value, varCount[0]);
173                         tokenList.add(new Token(value, "Переменная", value +
    ↪ ":" + varCount[0]++));
174                     }
175                 } else {
176                     tokenList.add(new Token(value));
177                 }
178                 break;
179         }
    }

```

```
180     }
181 }
182
183 private static boolean isNumeric(String str) {
184     try {
185         Double.parseDouble(str);
186         return true;
187     } catch (NumberFormatException e) {
188         return false;
189     }
190 }
191
192 static class Token {
193     String value;
194     String type;
195     String tokenValue;
196
197     Token(String value) {
198         this.value = value;
199         this.type = "Неизвестный";
200         this.tokenValue = "";
201     }
202
203     Token(String value, String type, String tokenValue) {
204         this.value = value;
205         this.type = type;
206         this.tokenValue = tokenValue;
207     }
208
209     String getType() {
210         return type;
211     }
212
213     String getValue() {
214         return tokenValue;
215     }
216 }
217 }
```
